



SQL DDL: Criando o Esquema

Aula 13 · Bloco 4 — SQL e Projeto Físico

Do esquema relacional ao banco rodando

Nesta aula

1. Do esquema relacional ao **script**
2. **Tipos de dados** do PostgreSQL
3. As **restrições** — e onde cada uma nasce
4. **ON DELETE** e **ON UPDATE** na prática
5. **Lendo os erros** do PostgreSQL

Do esquema relacional ao script

OBRA(cod_obra, titulo, ano, isbn) vira:

```
CREATE TABLE obra (  
    cod_obra  SERIAL PRIMARY KEY,  
    titulo    VARCHAR(200) NOT NULL,  
    ano       INTEGER      CHECK (ano BETWEEN 1450 AND 2100),  
    isbn      CHAR(13)      UNIQUE  
);
```

Cada linha do **domínio** que você escreveu na aula 04 virou uma restrição.

Tipos de dados

Tipo	Use para	Tipo	Use para
INTEGER	inteiros	VARCHAR(n) · TEXT	texto
NUMERIC(p, s)	dinheiro	CHAR(n)	tamanho fixo
REAL	medidas científicas	DATE · TIMESTAMP	data e hora
BOOLEAN	verdadeiro/falso	SERIAL	chave automática

⚠ **Dinheiro nunca em REAL** . Ponto flutuante arredonda, e o cliente confere o centavo.

Chave artificial autoincrementada

```
cod_obra SERIAL PRIMARY KEY           -- forma clássica  
cod_obra INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY -- padrão SQL
```

O banco gera o próximo número sozinho, sem risco de colisão entre sessões concorrentes.

💡 E lembre da aula 06: chave artificial **não dispensa** a natural. `cod_obra SERIAL PRIMARY KEY` e `isbn UNIQUE`.

As cinco restrições

```
CREATE TABLE emprestimo (  
    id          SERIAL PRIMARY KEY,           -- entidade  
    matricula   INTEGER NOT NULL             -- obrigatório  
                                REFERENCES usuario(matricula), -- referencial  
    data_retirada DATE NOT NULL DEFAULT CURRENT_DATE, -- valor padrão  
    data_prevista DATE NOT NULL,  
    CHECK (data_prevista > data_retirada)      -- semântica  
);
```

PRIMARY KEY · FOREIGN KEY · UNIQUE · NOT NULL · CHECK

Chave composta e entidade fraca

A entidade fraca da aula 06 vira exatamente isto:

```
CREATE TABLE exemplar (  
    cod_obra      INTEGER REFERENCES obra(cod_obra),  
    num_exemplar  INTEGER,  
    situacao      VARCHAR(20) NOT NULL,  
    PRIMARY KEY (cod_obra, num_exemplar)      -- chave do pai + parcial  
);
```

A PK composta é a **tradução literal** do que o DER dizia.

ON DELETE na prática

```
matricula INTEGER REFERENCES usuario(matricula) ON DELETE RESTRICT,  
cod_obra  INTEGER REFERENCES obra(cod_obra)      ON DELETE CASCADE
```

- **RESTRICT** — não deixa apagar o usuário que tem empréstimo. O padrão seguro;
- **CASCADE** — apagar a obra apaga os exemplares junto;
- **SET NULL** — a FK vira nula. A coluna precisa aceitar.

⚠ Escolha **por relacionamento**, com a pergunta: *"o filho faz sentido sem o pai?"*

ALTER TABLE, DROP — e a ordem

```
ALTER TABLE obra ADD COLUMN edicao INTEGER;  
ALTER TABLE obra ALTER COLUMN titulo TYPE VARCHAR(300);  
ALTER TABLE obra ADD CONSTRAINT ck_ano CHECK (ano > 1450);
```

```
DROP TABLE emprestimo;           -- filhos PRIMEIRO  
DROP TABLE usuario;             -- pais DEPOIS
```

⚠ Criar é do pai para o filho. **Apagar é do filho para o pai.** Inverter dá `violates foreign key constraint` — e o script para no meio.

Lendo os erros do PostgreSQL

Mensagem	O que aconteceu
<code>relation "x" does not exist</code>	tabela não criada, ou nome errado
<code>violates foreign key constraint</code>	FK apontando para PK inexistente
<code>duplicate key value violates unique</code>	valor repetido em PK ou UNIQUE
<code>null value in column violates not-null</code>	faltou valor obrigatório
<code>syntax error at or near</code>	erro de digitação — olhe a linha de cima

Exercícios da aula

Na pasta `aula-13/` :

1. `ex01.sql` — o `CREATE TABLE` de três entidades do seu modelo, com todas as restrições;
2. `ex02.sql` — a entidade fraca com PK composta;
3. `ex03.sql` — provoque os cinco erros da tabela e copie cada mensagem num comentário;
4. `ex04.sql` — `ALTER TABLE` para três mudanças de esquema;
5. **Desafio** 🌶️ `ex05.sql` — o script completo do seu minimundo, que roda **do zero** sem erro.

Próxima aula

Aula 14 — SQL DML e Consultas

Povoar o banco — e fazer as perguntas
que motivaram o curso inteiro.